

Between Lines of Code: Unraveling the Distinct Patterns of Machine and Human Programmers

Yuling Shi

Shanghai Jiao Tong University
yuling.shi@sjtu.edu.cn

Hongyu Zhang

Chongqing University
hyzhang@cqu.edu.cn

Chengcheng Wan

East China Normal University
ccwan@sei.ecnu.edu.cn

Xiaodong Gu*

Shanghai Jiao Tong University
xiaodong.gu@sjtu.edu.cn

Abstract—Large language models have catalyzed an unprecedented wave in code generation. While achieving significant advances, they blur the distinctions between machine- and human-authored source code, causing integrity and authenticity issues of software artifacts. Previous methods such as DetectGPT have proven effective in discerning machine-generated texts, but they do not identify and harness the unique patterns of machine-generated code. Thus, its applicability falters when applied to code. In this paper, we carefully study the specific patterns that characterize machine- and human-authored code. Through a rigorous analysis of code attributes such as lexical diversity, conciseness, and naturalness, we expose unique patterns inherent to each source. We particularly notice that the syntactic segmentation of code is a critical factor in identifying its provenance. Based on our findings, we propose DetectCodeGPT, a novel method for detecting machine-generated code, which improves DetectGPT by capturing the distinct stylized patterns of code. Diverging from conventional techniques that depend on external LLMs for perturbations, DetectCodeGPT perturbs the code corpus by strategically inserting spaces and newlines, ensuring both efficacy and efficiency. Experiment results show that our approach significantly outperforms state-of-the-art techniques in detecting machine-generated code.¹

Index Terms—machine-generated code detection, large language models, code generation, empirical analysis

I. INTRODUCTION

The advent of large language models (LLMs) such as Codex [1] and ChatGPT [2] has revolutionized software engineering tasks such as code generation. Through extensive training on ultra-large code corpora [3], [4], [5], [6], LLMs acquire the ability to generate syntactically and functionally correct code, bringing a new era of efficiency and innovation in software creation, maintenance, and evolution.

While capable of generating human-like code, LLMs bring ambiguity of whether a software artifact is created by human or machine, causing integrity and authenticity issues in software development. This indistinction can lead to various challenges, such as misattribution of code ownership for bug triage and inflated assessments of developer workloads. Potential vulnerabilities in machine-generated code may go unnoticed due to the overreliance on its perceived robustness. The blending of human and machine efforts not only raises questions about the trustworthiness of the software but also threatens the

integrity of development process, wherein the true authorship and the effort invested in creating software artifacts become obscured. Addressing these concerns is pivotal in maintaining a transparent and secure software development lifecycle.

Recently, there has been a growing research trend in detecting machine-generated texts [7], [8]. Perturbation-based methods like DetectGPT [9] have achieved state-of-the-art results in identifying machine-generated text. These methods employ likelihood score discrepancies between the original text and its various LLM-perturbed variants for detection, capturing the distinct patterns of machine-generated text that machines tend to prefer to a smaller set of phrases and expressions. However, such detection methods tailored for natural language texts face challenges when applied to code, as code requires strict adherence to syntactic rules, while natural language can maintain coherence with more variation [10]. This situation highlights a significant gap in existing research: a lack of in-depth assessment of the intrinsic features of machine- and human-authored code, crucial for understanding the unique patterns of machine-generated code and devising effective detection methods.

In this paper, we conduct a comparative analysis of the distinct patterns between machine- and human-authored code from three aspects, including lexical diversity, conciseness, and naturalness. Through our analysis, we uncover that compared to human, machine tends to write more concise and natural code with a narrower spectrum of tokens and adhere to common programming paradigms, and the disparity is more pronounced in stylistic tokens such as the whitespace tokens.

Based on the findings, we propose a novel method called DetectCodeGPT for detecting machine-authored code. We extend the perturbation-based framework of DetectGPT by strategically inserting stylistic tokens, such as whitespace and newline characters, to capture the distinct patterns between machine- and human-authored code. This approach capitalizes on our observation that the disparity in coding styles is more pronounced in these stylistic tokens. By directly manipulating the code, DetectCodeGPT eliminates the need for an external pre-trained model, thereby enhancing both efficiency and effectiveness in the detection process.

To evaluate the effectiveness of DetectCodeGPT, we have conducted extensive experiments on two datasets across six code language models. The results demonstrate that DetectCodeGPT significantly outperforms the state-of-the-art methods

¹Code available at <https://github.com/YerbaPage/DetectCodeGPT>

* Xiaodong Gu is the corresponding author

by 7.6% in terms of AUC. Moreover, it proves to be a model-free and robust method against model discrepancies, making it viable for real-world applications with unknown or inaccessible source models.

Our contributions can be summarized as follows:

- To our knowledge, we are the first to conduct a comprehensive and thorough analysis of the distinct patterns of LLM-generated code. Our study sheds light on essential insights that can further advance the utility of LLMs in programming.
- We propose a novel method for detecting machine-generated code by leveraging its distinct stylistic patterns.
- We extensively evaluate the DetectCodeGPT across a variety of settings and show the effectiveness of our approach.

II. BACKGROUND

A. Large Language Models for Code

Large language models [11], [12] based on Transformer [13] decoder has achieved remarkable success in natural language processing tasks [14]. In the domain of code generation, Codex [1] and AlphaCode [15] are pioneering works to train large language models on code. The training data often contain millions of code in different programming languages collected from open source repositories like GitHub [16], [17], [18]. Later advances to improve LLMs on code include designing new pretraining tasks like fill-in-the-middle [19], [20], [4], [5] and also instruction fine-tuning [21], [6]. Recent large language models pretrained on a mixture of programming and natural languages like ChatGPT [22] and LLaMA [23] have also shown promising results on code generation tasks.

B. Perturbation-Based Detection of Machine-Generated Text

In the realm of machine-generated text detection, perturbation based method like DetectGPT [9] stands as the state-of-the-art technology [7]. In this section, we take DetectGPT as an example to illustrate the idea of perturbation-based detection methods. DetectGPT distinguishes between machine and human-generated text by analyzing the patterns in their probability scores [9]. The core idea is that when a text x generated from a machine is subtly changed to $\tilde{x}$ through a perturbation process $q(\cdot|x)$ (e.g., MLM with T5 [24]), there is a sharper decline in its log probability scores $\log p_\theta(x)$ than that in human-generated text. This is because a machine-generated text is usually more predictable and tightly bound to the patterns it was trained on, leading to a distinct negative curvature in log probability when the text is perturbed. By contrast, human-written texts are characterized by a rich diversity that reflects a blend of experiences and cognitive processes. As a result, it doesn't follow such predictable patterns, and its log probability scores $\log p_\theta(x)$ do not plummet as dramatically when similarly perturbed. Based on such discrepancy, we can define a *likelihood discrepancy score* for each input code to measure the drop of log probability after perturbation.

$$\mathbf{d}(x, p_\theta, q) \triangleq \log p_\theta(x) - \mathbb{E}_{\tilde{x} \sim q(\cdot|x)} \log p_\theta(\tilde{x}) \quad (1)$$

Table I: Studied categories of Python code tokens

Category	Tree-sitter Types
keyword	def, return, else, if, for, while, ...
identifier	identifier, type_identifier
literal	string_content, integer, true, false, ...
operator	<, >, =, ==, +, ...
syntactic symbol	:,),], [, (, /, ", ', {, }, .
comment	comment
whitespace	space, \n

By inspecting these scores, we can detect the source of x . A significant drop indicates machine authorship and a smaller change suggests a human creator. This method effectively captures the more nuanced and variable nature of human-generated text compared to the more formulaic and patterned output of language models.

III. EMPIRICAL ANALYSIS

In this section, we conduct a comparative analysis of the distinct features of machine- and human-authored code.

A. Study Design

To gain insights into distinctions between human and machine programmers, we consider three primary aspects that are relevant to coding styles, namely diversity, conciseness, and naturalness [25], [26], [27], [28], [29], which can be measured by specific metrics.

1) *Lexical Diversity*: Lexical diversity indicates the richness and variety of vocabulary present in a corpus. In the context of programming, this refers to the diversity in variable names, functions, classes, and reserved words. Analyzing lexical diversity offers a deeper understanding of the creativity, expressiveness, and potential complexity of code segments. There are four important empirical metrics in both natural and programming languages revealing the lexical diversity: token frequency, syntax element distribution, Zipf's law [30] and Heaps' law [31].

Token Frequency stands for the occurrence of distinct tokens in the code corpus. The attribute indicates the core vocabulary utilized by human and machine programmers, shedding light on their coding preferences and tendencies.

Syntax Element Distribution refers to the proportion of syntax elements (e.g., keywords, identifiers) in the code corpus. Understanding the distribution of syntax elements in code is akin to dissecting the anatomy of a language. It gives us a lens to view the nuances of coding style, the emphasis on structure, and the intricacies that distinguish human- and machine-authored code.

To delve into the syntax element distribution, we analyze code with tree-sitter² and classify tokens into distinct categories, as detailed in Table I. We then compute the proportion of each category in the code corpus.

Zipf's and Heaps' Laws were initially identified in natural languages [30], [31], and later verified in the scope of programming languages [25], [26]. Zipf's law states that the frequency value f of a token is inversely proportional to its

²<https://github.com/tree-sitter/tree-sitter>

Table II: Top 50 tokens from human- and machine-authored code from CodeLlama

Rank	Human-Authored Tokens	Machine-Authored Tokens
1–10	. () = ' , : self " [	. - , () self : " ' if
11–20	] if return in for not None 0 1 ==	= return not [def raise isinstance] == path
21–30	else + is name { } path data raise -	name 0 __class__ __name__ None os { / } %
31–40	try * os len format get and True value isinstance	else TypeError str ' __init__ is > // the
41–50	args key % np i x kwargs except False or	in 1 ; value kwargs #include + __str__ for ValueError

frequency rank r : $f \propto \frac{1}{r^\alpha}$, where α is close to 1 [30]. In programming languages, it states that a few variable names or functions are very commonly used across different scripts, while many others are rarely employed. Heaps' Law characterizes the expansion of a vocabulary V as a corpus D increases in size: $V \propto D^\beta$, where $\beta \in (0, 1)$ captures the rate of vocabulary growth relative to the size of the corpus.

We investigate how closely machine-authored code aligns with Zipf's and Heaps' laws compared to human-authored code, which could reflect the models' ability to mimic human's lexical usage.

2) *Conciseness*: Conciseness stands as a cornerstone attribute when characterizing code [32], [28], [29]. The intricate balance of code conciseness directly influences readability, maintainability, and even computational efficiency. We investigate two metrics that characterize code conciseness, namely, the number of tokens and the number of lines.

Number of tokens gives us an indication of verbosity and complexity, showing the detailed composition of the code [32].

Number of lines helps us understand organizational choices, as spreading code across more lines can reflect a focus on readability and structure [29].

3) *Naturalness*: The concept of code naturalness suggests that programming languages share a similar degree of regularity and predictability with natural languages [27]. This idea has been operationalized by employing language models to assess the probability of a specific token's occurrence within a given context. Under this framework, we inspect how "natural" machine-generated code is compared to human-written code.

Token Likelihood and Rank are two metrics that measure the naturalness of each token in the studied code corpus. The token likelihood stands for the probability p of a token x under the model p_θ , denoted as $p_\theta(x)$. The rank of a token x is the position of x in the sorted list of all tokens based on their likelihoods, denoted as $r_\theta(x)$. Both metrics evaluate how likely a token is preferred by the model [33], [34], [35]. We calculate log scores on each token and then take the average to represent the whole code snippet as advised in [34]. To pinpoint the code elements that most significantly affect the score discrepancies, we also present the mean scores on different syntax element categories in Table I for comparison.

B. Experimental Setup

We choose the state-of-the-art CodeLlama model [6] to generate code. Limited by our computational resources, we use the version with 7B parameters. As for the decoding strategies, we adopt the top- p sampling method [36] with $p=0.95$ following [1]. The temperature T is an important

parameter controlling the diversity of the generated code [36]. Since current LLMs on code are usually evaluated across different decoding temperatures [5], [37], [1], [6], we generate code with $T = 0.2$ and $T = 1.0$ to capture the model's behavior under different settings. The maximum length of the generated code is set to 512 tokens based on the memory constraints and the length distribution of human-written code. All experiments are conducted on 2 NVIDIA RTX 4090 GPUs with 24GB memory.

C. Dataset Preparation

To compare with human-authored code, we extract 10,000 Python functions randomly from the CodeSearchNet corpus [16], which is curated from a wide range of open-source GitHub projects. We use the function signatures and their accompanying comments as prompts for the model as in [1]. We also collect the corresponding bodies of these functions to represent human-written code.

While acknowledging that current models, including CodeLlama and even ChatGPT [38], may not yet craft code of unparalleled quality for intricate tasks such as those in CodeSearchNet [39], [38], the choice of this dataset is deliberate and insightful. Challenging the models against various real-world project code rather than simple programming problems, akin to those in the HumanEval [1] or MBPP [40] dataset, offers a more representative assessment. It allows us to analyze the differences between human- and machine-authored code when faced with broader, practical applications.

D. Results and Analysis

We present the results and analysis regarding each code attribute introduced in Section III-A.

1) *Token Frequency*: Table II lists the top 50 tokens from human- and machine-authored code when $T=0.2$. Due to space limit, we omit the results when $T=1.0$, which has a similar result. From the results, we have several noteworthy observations:

Common Tokens: Human- and machine-authored code shares a commonality in their usage of certain tokens, including punctuation marks such as ":", "(", ")", and structural keywords such as "if", "return", and "else". This is because LLMs acquire foundational coding syntax after being trained on extensive human-written code corpora.

Error Handling: Tokens associated with error handling like "raise" and "TypeError", are more prevalent in machine-authored code. This difference implies that machine programmers emphasize robustness and exception handling more explicitly.

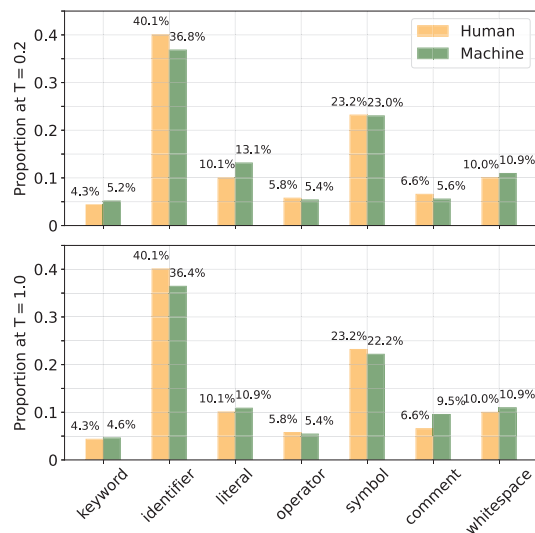


Figure 1: Syntax element distribution of the code corpus

Programming Paradigms: Tokens indicating object-oriented programming like “self” and “__init__” are prominent in both human- and machine-authored code, which illustrates the model’s training alignment with this paradigm. However, machine-authored code appears to favor more boilerplate tokens like “__class__” and also “__name__”, which could stem from its training on diverse object-oriented codebase.

Finding 1: Machine-authored code pays more attention to exception handling and object-oriented principles than human, suggesting an emphasis on error prevention and adherence to common programming paradigms.

2) Syntax Element Distribution: The analysis of syntax element distributions, as visualized in Figure 1, reveals intriguing insights into the coding conventions and stylistic nuances between human- and machine-authored code. The “keyword”, “operator”, “syntactic symbols”, and “whitespace” proportions remain largely consistent between human and machine-authored code, across both temperatures. This consistency suggests that the foundational syntactical elements manifest similarly in both datasets. Delving deeper into the more nuanced discrepancies, a few categories emerge that underscore the differential preferences or tendencies of human and machine writers (statistical significant from Chi-square test with $p < 0.01$):

Identifier: The identifiers constitute a significantly lower proportion among machine-authored code across both temperatures, indicating that machine-authored code may have a more compact style with fewer identifiers.

Literal: Machine-authored code consistently shows a slightly higher tendency towards using literals. Across both temperatures, the machine code exhibits an increase in the literal proportion compared to the human-written code. This suggests that machine-authored code may be more likely to process raw data directly. This could result from the machine’s training on diverse data manipulation tasks.

Comment: Machine-authored code has much more comments when $T=1.0$. This observation hints machine’s increased emphasis on code documentation and explanation with higher temperatures, when it becomes less deterministic and more exploratory.

Finding 2: Machine-authored code tends to use fewer identifiers, more literals for direct data processing, and have more comments when the generation temperature grows.

3) Zipf’s and Heaps’ Laws: Figure 2 offers a comprehensive understanding of coding tendencies of both human and machine programmers. Starting with Zipf’s law, Figure 2a and 2b both delineate similar trends for human and machine programmers, corroborating the law’s applicability. And we can observe machine’s heightened proclivity towards tokens ranked between 10 and 100, especially at $T=0.2$. Turning our attention to Heaps’ law, the near-linear trends in Figure 2c-2d reaffirm the law’s validity. Also, there’s a noticeable shallowness in the slope for machine’s code at $T=0.2$ revealing machine’s decreased lexical diversity.

The obvious differences at $T=0.2$ can be ascribed to *human creativity and variability*. The varied approaches and methodologies humans employ can lead to a diversified token usage within this range. Another plausible interpretation can be its *risk aversion*. A machine, especially at lower temperatures, might be reverting to familiar patterns ensuring correctness in code generation. Additionally, certain patterns within the training data might have been overemphasized due to *model overfitting*, leading the machine to a skewed preference.

Finding 3: Machines demonstrate a preference for a limited spectrum of frequently-used tokens, whereas human code exhibits a richer diversity in token selection.

4) Number of Tokens and Lines: Figure 3 presents the distribution of code length under different settings. For the temperature setting of 0.2, machine-authored code exhibits more conciseness, both in token and line numbers. As the temperature increases to $T = 1.0$, we witness a convergence of distributions. The gap narrows, yet the machine’s preference for relatively concise code persists. This reveals that higher temperatures induce more exploratory generative behavior in the model, leading to diverse coding styles.

One could hypothesize several reasons for these observed patterns. The propensity for conciseness at lower temperatures may reflect LLM’s training data, where probably concise solutions were more prevalent or deemed more “correct”. On the flip side, human developers, often juggling multiple considerations like future code extensions, comments for peer developers, or even personal coding style, might craft lengthier solutions. Furthermore, the narrowing of disparities at higher temperatures can be attributed to the model’s increased willingness to explore varied coding styles. At higher temperatures, the LLM possibly mimics a broader spectrum of human coding patterns, capturing the essence of diverse coding habits and styles found in its training corpus.

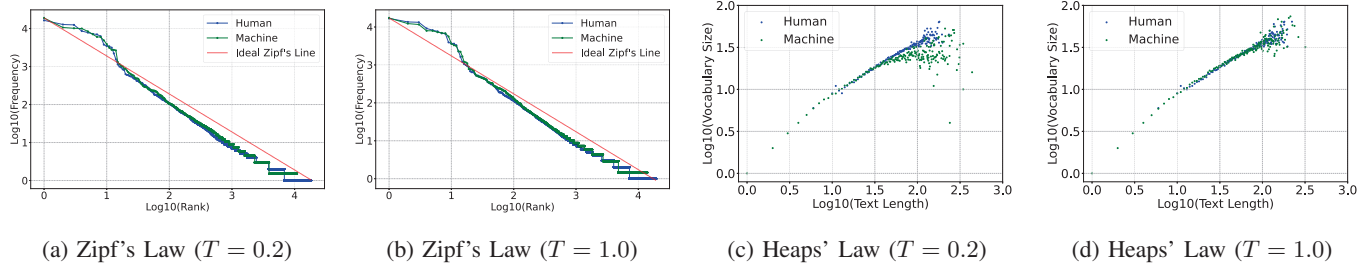


Figure 2: Comparison of Zipf's and Heaps' laws on machine- and human-authored code

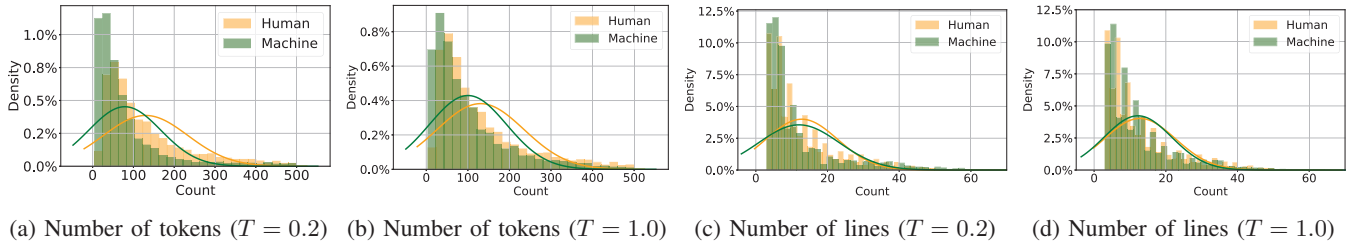


Figure 3: Distribution of code length for machine- and human-authored code

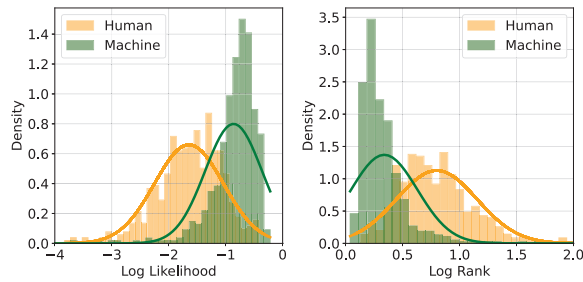


Figure 4: Distribution of naturalness scores

Finding 4: Machines tend to write more concise code as instructed by their training objective, while human programmers tend to write longer code, reflective of their stylistic preferences.

5) *Token Likelihood and Rank:* Figure 4 shows that there is a great discrepancy of naturalness between machine- and human-authored code. Compared to human-authored code, the log likelihood scores of machine-authored code are mostly higher and the log rank scores are mostly lower, indicating that machine's code is more "natural" than human-written code. Such observation in source code is consistent with the findings in natural language [34], [33], [9], [35].

Table III summarises the comparison results in terms of each token category at $T = 0.2$. An intriguing finding is that whitespace tokens stand out with the highest deviation of naturalness, surpassing even the combined use of all tokens. This highlights a distinctive aspect of coding styles: machines, trained on extensive datasets, typically generate code with regular, predictable whitespace patterns. Humans, however, influenced by individual styles and practices, exhibit a wider variety in their use of whitespaces. The distinct patterns in

Table III: The naturalness of different categories of syntax elements. Statistical significance $p < 0.001$.

Category	Log Likelihood			Log Rank		
	Machine	Human	Δ	Machine	Human	Δ
keyword	-1.701	-2.128	0.428	0.837	1.053	0.217
identifier	-0.459	-0.874	0.415	0.163	0.378	0.215
literal	-0.506	-1.364	0.858	0.152	0.630	0.479
operator	-0.938	-1.835	0.897	0.367	0.872	0.504
symbol	-0.868	-1.639	0.771	0.321	0.781	0.460
comment	-1.503	-3.028	1.525	0.608	1.610	1.002
whitespace	-1.131	-2.740	1.609	0.429	1.441	1.012
ALL	-0.827	-1.658	0.831	0.319	0.811	0.492

machine-generated whitespaces, therefore, point to an inherent variation in coding style between machine and human.

Finding 5: Machine-authored code exhibits higher "naturalness" than human-authored code, and the disparity is more pronounced in tokens such as comments, and whitespaces, which are more reflective of individual coding styles.

IV. DETECTING MACHINE GENERATED CODE

The empirical results suggest that machines tend to write more concise and natural code with a narrower spectrum of tokens, adhering to common programming paradigms, and the disparity is more pronounced in stylistic tokens such as whitespaces. This sparks a new idea for detecting machine-authored code: instead of perturbing arbitrary tokens, we focus on perturbing those stylistic tokens that best characterize the machine's preference. Based on this idea, we introduce DetectCodeGPT, a novel zero-shot method for detecting machine-authored code.

A. Problem Formulation

We formulate the detection of machine-authored code as a classification task, which predict whether or not a given

code snippet x is produced by a source model p_θ . For this purpose, we transform x to an equivalent form $\tilde{x}$ through a perturbation process $q(\cdot|x)$. We anticipate a sharper decline in its naturalness score if x is written by an LLM. The key problems here are how to define the naturalness score and how to design the perturbation process. We introduce the naturalness score and the perturbation strategy $q(\cdot|x)$ in our approach in the following sections.

B. Measuring Naturalness

Previous methods usually use the log likelihood of tokens to measure the naturalness of machine-authored content [27], [35]. However, the log rank of tokens shows better performance comparing the naturalness of machine- and human-authored text [9], [34], because it offers a smoother and robust representation of token preference.

Unlike DetectGPT which directly calculates the log likelihood of tokens, we adopt the Normalized Perturbed Log Rank (NPR) score [41] to capture the naturalness. The NPR score is formally defined as:

$$\text{NPR}(x, p_\theta, q) \triangleq \frac{\mathbb{E}_{\tilde{x} \sim q(\cdot|x)} \log r_\theta(\tilde{x})}{\log r_\theta(x)}, \quad (2)$$

where $\log r_\theta(x)$ is the logarithm of the rank order of text x sorted by likelihood under model p_θ . In practice, $\text{NPR}(x, p_\theta, q)$ has been demonstrated to be more accurate for differentiating text origins, outperforming the log likelihood discrepancy $\mathbf{d}(x, p_\theta, q)$ [41].

C. Perturbation Strategy

Our empirical study indicates that the whitespace tokens serve as an important indicator of machine's regularization and human's diversity, which points to an inherent variation in coding style. Therefore, we propose an efficient and effective perturbation strategy with the following two types of perturbations below. Detailed explanations on the effectiveness of these perturbations are given in Section VI-A.

1) *Space Insertion*: Let C represent the set of all possible locations to insert spaces in a code segment. We randomly select a subset $C_s \subseteq C$ such that $|C_s| = \alpha \times |C|$, where $\alpha \in [0, 1]$ is a fraction representing the code locations. For each location $c \in C_s$, we introduce a variable number of spaces, $n_{\text{spaces}}(c)$, which is drawn from a Poisson distribution $\mathcal{P}(\lambda_{\text{spaces}})$. The Poisson distribution is chosen to simulate the randomness in human coding styles, similar to the random text infilling strategy in BART [42]. Mathematically, this can be represented as:

$$n_{\text{spaces}}(c) \sim \mathcal{P}(\lambda_{\text{spaces}}). \quad (3)$$

2) *Newline Insertion*: We split the code into lines and obtain a set L of lines. A subset $L_n \subseteq L$ is then chosen randomly, where $|L_n| = \beta \times |L|$, with $\beta \in [0, 1]$ denoting the proportion of the line locations. For each line $l \in L_n$, we introduce a variable number of newlines, $n_{\text{newlines}}(l)$, also sampled from a Poisson distribution $\mathcal{P}(\lambda_{\text{newlines}})$:

Algorithm 1: DetectCodeGPT: Machine-Generated Code Detection with Stylized Code Perturbation

Data: code x , source model $\mathcal{M}$, number of perturbations k , decision threshold ϵ , parameters $\alpha, \beta, \lambda_{\text{spaces}}$, and $\lambda_{\text{newlines}}$

```

1 for  $i \leftarrow 1$  to  $k$  do
    // Random decision for type of
    // perturbation
2    $p \sim \mathcal{U}(0, 1)$ ;
3   if  $p \leq 0.5$  then
4     // Spaces Insertion
5     Let  $C$  represent all possible locations to insert spaces
        in  $x$ ;
6     Select  $C_s \subseteq C$  such that  $|C_s| = \alpha \times |C|$ ;
7     for each location  $c \in C_s$  do
8        $n_{\text{spaces}}(c) \sim \mathcal{P}(\lambda_{\text{spaces}})$ ;
9       Insert  $n_{\text{spaces}}(c)$  spaces at location  $c$  in  $x$ ;
10    end
11  else
12    // Newlines Insertion
13    Split the perturbed code  $x$  into a set  $L$  of lines;
14    Select  $L_n \subseteq L$  such that  $|L_n| = \beta \times |L|$ ;
15    for each line  $l \in L_n$  do
16       $n_{\text{newlines}}(l) \sim \mathcal{P}(\lambda_{\text{newlines}})$ ;
17      Insert  $n_{\text{newlines}}(l)$  newlines after line  $l$  in  $x$ ;
18    end
19  end
20  Store the perturbed code as  $\tilde{x}_i$ ;
21 end
22 Estimate NPR:
23    $\text{NPR}_x \leftarrow \text{NPR}(x, p_\theta, q) - \frac{1}{k} \sum_i \text{NPR}(\tilde{x}_i, p_\theta, q)$ ;
24   if  $\text{NPR}_x > \epsilon$  then
25     return true; // Probably machine-authored
26   else
27     return false; // Probably human-authored
28 end

```

$$n_{\text{newlines}}(l) \sim \mathcal{P}(\lambda_{\text{newlines}}). \quad (4)$$

We randomly choose one type of perturbation to the code snippet x to generate a set of perturbed samples $\tilde{x}_i$ for $i \in [1, k]$, where k is the number of perturbations. Through this step, we instill randomness at a granular stylistic level, thereby amplifying the perturbation's efficacy. Our perturbation strategy introduces several distinct advantages over the conventional methods [9], [41] using MLM to perturb the code, which will be discussed in Section VI-B.

Algorithm 1 summarizes the entire workflow of DetectCodeGPT. Our algorithm harnesses stylized code perturbation to differentiate between human- and machine-authored code. At the core of our approach is the strategic insertion of spaces (Lines 4-9) and newlines (Lines 11-16) in code, a process that simulates the inherent randomness in human coding styles. The algorithm operates by generating perturbed versions of the code and then evaluating their NPR scores (Lines 20-21) with respect to the source model $\mathcal{M}$.

The threshold parameter ϵ in Line 22, pivotal for making the detection decision, offers flexibility in catering to different application scenarios. By adjusting ϵ , users can balance between

false positives and false negatives, tailoring the detection sensitivity according to the specific needs of the deployment context.

V. EVALUATION

We conduct experiments to evaluate the effectiveness of DetectCodeGPT, aiming to answer the following research questions.

- **RQ1:** How effectively does our method distinguish between machine-generated and human-written code?
- **RQ2:** To what extent do individual components influence the overall performance of our method?
- **RQ3:** What is the impact of varying the number of perturbations on the detection performance?
- **RQ4:** How effective is our method in cross-model code detection?

A. Datasets

We carefully use a different split of the *CodeSearchNet* dataset [16] from the one used in the empirical study for evaluation. We select Python code from *The Stack* [17] as another evaluation dataset. Similar to *CodeSearchNet*, *The Stack* provides code from a variety of open-source projects representative of real-world scenarios. We use a parsed and filtered version [43] of this dataset and also concatenate the function definitions with their corresponding comments as prompts as in [1]. For each combination of dataset and model, we sample 500 human and machine code pairs for evaluation. The maximum length of code is trimmed to 128 tokens.

B. Studied Models

We investigate machine-generated code by a diverse array of advanced LLMs, including InCoder [20], Phi-1 [44], StarCoder [45], WizardCoder [21], CodeGen2 [4] and CodeLlama [6]. We obtain their checkpoints from Huggingface³ with different parameter sizes (1B-7B).

C. Evaluation Metric

Following prior works [9], [41], our primary metric for performance evaluation is the Area Under the Receiver Operating Characteristic curve (AUROC). Formally, given a set of true positive rates (TPR) and false positive rates (FPR) across different thresholds, the AUROC can be represented as:

$$\text{AUROC} = \int_0^1 \text{TPR}(t) dt, \quad (5)$$

where t denotes varying threshold values. It provides a comprehensive view of performance across all possible thresholds, making it threshold-independent. This makes the metric both interpretable and insightful, offering a clearer picture of the model's discriminating capabilities.

³<https://huggingface.co/models>

D. Baselines

Our evaluation is benchmarked against a diverse range of zero-shot machine-generated text detection techniques. And a supervised baseline is also included to demonstrate the advantages of our zero-shot method:

- **Log $p(x)$** [35]: Utilizes the source model's average token-wise log probability to gauge code naturalness. Machine-generated code tends to have a higher score.
- **Entropy** [34]: Interprets high average entropy in the model's predictive distribution as indicative of machine generation.
- **(Log-) Rank** [34], [33]: The average observed rank or log rank of each token in the LLM prediction, with machine-generated passages typically showing smaller average values.
- **DetectGPT** [9]: Leverages the log probability of the original code and its perturbed variants to compute the perturbation discrepancy gap.
- **DetectLLM** [41]: Introduces two methods, one blends log likelihood with log rank to compute **LRR**, and the other improves DetectGPT by incorporating the **NPR** score.
- **GPTSniffer** [46]: A supervised baseline that trains CodeBERT [47] to predict the authorship of a given code snippet. Following OpenAI's RoBERTa-based [48] GPT detector⁴, we train the model on a combination of 1000 samples generated by each model at each setting.

E. Experimental Setup

For code generation with different models, we adopted the top- p sampling strategy [36] with $p=0.95$ following [1]. We explored two temperature settings, $T=0.2$ and $T=1.0$, as discussed in Section III-B. The maximum length constraint for generated code was set at 128 tokens. With respect to the perturbation-specific hyperparameters, a grid search on a held-out set from the *CodeSearchNet* dataset, using the SantaCoder model [3], revealed optimal values. Consequently, we set α and β to 0.5, while $\lambda_{\text{spaces}}=3$ and $\lambda_{\text{newlines}}=2$. For all experiments, we maintained a consistent configuration of generating 50 perturbations.

For the DetectGPT and DetectLLM, which involve an LLM in restoring perturbed code, we utilized the CodeT5+ (770M) model [49]. And as for the supervised baseline GPTSniffer, we trained the CodeBERT model for 5 epochs with a batch size of 16 and a learning rate of $2e-5$ with AdamW optimizer. All experiments are conducted on 2 NVIDIA RTX 4090 GPUs with 24GB memory.

F. Detection Performance (RQ1)

Table IV delineates the results of various methods. According to the results, DetectCodeGPT consistently outperforms baseline methods. Compared to the strongest baseline Log Rank, our method achieves an average relative improvement of 7.6% in AUROC. In an impressive 20 of 24 combinations of dataset and model, our method provides the most accurate performance, which underscores its robustness across a variety

⁴<https://github.com/openai/gpt-2-output-dataset/tree/master/detector>

Table IV: Performance (AUROC) of various detection methods. Statistical significance $p < 0.001$.

Dataset	Code LLM	Detection Methods								
		$\log p(x)$	Entropy	Rank	Log Rank	DetectGPT	LRR	NPR	GPTSniffer	DetectCodeGPT
CodeSearchNet ($T = 0.2$)	Incoder (1.3B)	0.9810	0.1102	0.8701	0.9892	0.4735	0.9693	0.8143	0.9426	0.9896
	Phi-1 (1.3B)	0.7881	0.4114	0.6409	0.7513	0.7210	0.4020	0.7566	0.3855	0.8287
	StarCoder (3B)	0.9105	0.2942	0.7585	0.9340	0.6949	0.9245	0.9015	0.7712	0.9438
	WizardCoder (3B)	0.9079	0.2930	0.7556	0.9120	0.6450	0.7975	0.8677	0.7433	0.9345
	CodeGen2 (3.7B)	0.7028	0.4411	0.7328	0.7199	0.6051	0.7997	0.6177	0.5327	0.8802
	CodeLlama (7B)	0.8850	0.3174	0.7265	0.9016	0.8212	0.8332	0.5890	0.7496	0.9095
CodeSearchNet ($T = 1.0$)	Incoder (1.3B)	0.7724	0.4167	0.7797	0.7876	0.6258	0.7427	0.6801	0.6761	0.7882
	Phi-1 (1.3B)	0.6118	0.4588	0.5709	0.6299	0.7492	0.4528	0.7912	0.4158	0.8365
	StarCoder (3B)	0.6574	0.4844	0.6987	0.6822	0.6505	0.7050	0.6751	0.6299	0.6918
	WizardCoder (3B)	0.8319	0.3363	0.7273	0.8338	0.5972	0.6965	0.7516	0.7068	0.8392
	CodeGen2 (3.7B)	0.4484	0.6263	0.6584	0.4632	0.4797	0.5530	0.5208	0.4024	0.6798
	CodeLlama (7B)	0.6463	0.4855	0.6759	0.6656	0.6423	0.6768	0.6515	0.6442	0.7239
The Stack ($T = 0.2$)	Incoder (1.3B)	0.9693	0.1516	0.8747	0.9712	0.6061	0.9638	0.8571	0.9291	0.9727
	Phi-1 (1.3B)	0.8050	0.4318	0.6766	0.7622	0.7295	0.4022	0.8106	0.4640	0.8578
	StarCoder (3B)	0.9098	0.3077	0.7843	0.9329	0.6824	0.9135	0.9233	0.7715	0.9274
	WizardCoder (3B)	0.9026	0.3196	0.7963	0.9010	0.6385	0.7742	0.8574	0.7794	0.9243
	CodeGen2 (3.7B)	0.7171	0.4051	0.7930	0.7301	0.5288	0.7604	0.5670	0.4520	0.8513
	CodeLlama (7B)	0.8576	0.3565	0.7366	0.8793	0.8087	0.8358	0.5436	0.7619	0.8852
The Stack ($T = 1.0$)	Incoder (1.3B)	0.7310	0.4591	0.7673	0.7555	0.6124	0.7446	0.6787	0.6846	0.7833
	Phi-1 (1.3B)	0.7841	0.4205	0.6666	0.7475	0.6718	0.4106	0.7755	0.4984	0.8376
	StarCoder (3B)	0.6333	0.5025	0.7010	0.6609	0.5896	0.7080	0.6638	0.7243	0.6890
	WizardCoder (3B)	0.8293	0.3459	0.7484	0.8223	0.6377	0.6436	0.7929	0.7766	0.8384
	CodeGen2 (3.7B)	0.4816	0.6046	0.5631	0.4956	0.4337	0.5740	0.5178	0.4265	0.6595
	CodeLlama (7B)	0.5929	0.5260	0.6451	0.6091	0.6116	0.6365	0.6226	0.7494	0.6660
Average	-	0.7649	0.3961	0.7228	0.7724	0.6357	0.7050	0.7178	0.6507	0.8308

of generative models, ranging from the 1.3 billion parameter InCoder to the 7 billion parameter CodeLlama.

We also repeated the experiments 10 times and employed a Wilcoxon rank sum test to assess the statistical significance of the performance differences between the methods. Results show that the performance superiority of our method was statistically significant, with p-values less than 0.001. The high AUROC scores achieved across these diverse settings confirm the method’s superior capability to generalize and reliably differentiate between machine-generated and human-written code.

We can observe that the challenge of detection notably increases at a temperature setting of $T=1.0$ than $T=0.2$. This is possibly due to the higher randomness at this temperature, where models are likely to generate outputs with greater diversity in styles. Despite these increased difficulties, the proposed method maintains its leading position in detection accuracy.

It is worth mentioning that our zero-shot framework often outperforms the supervised GPTSniffer, highlighting the challenges of detecting machine-generated code with training data-dependent supervised models. The stable performance across various generation settings showcases our method’s advanced detection capabilities. This makes it an effective solution in practical applications.

To further illustrate the effectiveness of our approach, we present some representative examples by DetectCodeGPT and two most competitive baselines in Figure 5, using decision

thresholds ϵ set at the mean scores of all code snippets.

Example 1 is a machine-generated code snippet. DetectCodeGPT can correctly detect it, while the baselines fail. In this example, the first and last “if” statements are separated from the rest of the code with newlines, and the two “if” statements in the middle are modularized together since they have similar functionalities. Such modularization and separation of code blocks are captured by DetectCodeGPT thanks to the stylized perturbation. Example 2 and 3 are human-written code that are misclassified by other baselines, but DetectCodeGPT correctly identifies them. In Example 2, we can observe that the code blocks are sometimes separated with newlines (e.g., lines 5-7), but sometimes not (as seen with the rest of the code). In Example 3, although the code blocks are well separated with newlines, the human author omitted the spaces between operators “/” and “/”, but add spaces between “*” and “+”. Such freely and randomly stylized code reveals the inherent randomness in human coding habits. Baselines, relying solely on token-wise conditional distributions, struggle to capture the coding style’s randomness, whereas DetectCodeGPT effectively utilizes style information to discern between machine-generated and human-written code, demonstrating its prowess in detection through stylized code perturbation.

However, there are also cases where DetectCodeGPT fails. Example 4 is a human-written code misclassified as machine-generated by all the approaches. We can observe that this code snippet is well-structured with newlines and spaces. Its resemblance to machine-generated code is striking, posing a


```
import os
import re

if not os.path.isdir(path):
    raise ValueError('%s is not a directory' % path)

if not isinstance(include, (list, tuple)):
    include = [include]
if not isinstance(exclude, (list, tuple)):
    exclude = [exclude]

if not show_all:
    exclude.append(r'\.pyc$')
...

```

(a) Example 1

```
args_names = []
args_names.extend(function_spec.args)
if function_spec.varargs is not None:
    args_names.append(function_spec.args)

args_check = {}
for arg_name in arg_specs.keys():
    if arg_name not in args_names:
        args_check[arg_name] = self.check(
            arg_specs[arg_name], arg_name
        )
return args_check

```

(b) Example 2

```
save_test = random() > 0.8
audio = load_audio(fn)
num_chunks = len(audio)//chunk_size
listener.clear()

for i, chunk in enumerate(chunk_audio(audio, chunk_size)):
    print('\r' + str(i * 100./num_chunks) + '%')
    buffer = update(buffer[len(chunk):], chunk)
    conf = listener.update(chunk)

```

(c) Example 3

```
# If num is 0.
if (n == "0"):
    return 0

# Count sum of digits under mod 9
answer = 0
for i in range(0, len(n)):
    answer = (answer + int(n[i])) % 9

# If digit sum is multiple of 9, answer
# 9, else remainder with 9.
if (answer == 0):
    return 9
else:
    return answer % 9

```

(d) Example 4

Figure 5: Examples of machine- and human-authored code snippets with corresponding predictions.

Table V: Performance of different perturbation strategies

Perturb. Type	MLM	Newline	Space	Newline&Space
$T = 0.2$	0.5436	0.8703	0.8639	0.8852
$T = 1.0$	0.6226	0.6453	0.6504	0.6660

Table VI: Impact of varying the number of perturbations

#Perturbations	10	20	50	100	200
$T = 0.2$	0.6537	0.8825	0.8852	0.8855	0.8846
$T = 1.0$	0.5558	0.6584	0.6660	0.6660	0.6662

significant challenge for distinction. This example highlights the difficulty of detecting machine-generated code among well-structured human-written code with standard coding styles.

G. Ablation Study (RQ2)

In our ablation study, we compare the effectiveness of different perturbation strategies for detecting machine-generated code using the CodeLlama (7B) model on *The Stack* dataset. The results summarized in Table V illuminates the comparative advantage of our stylistic perturbation approach. We observe that both newline and space perturbations independently offer substantial improvements over the traditional MLM-based (CodeT5+) perturbation technique as in DetectGPT and DetectLLM [9], [41] for natural language. Also, the combination of newline and space perturbations further enhances the detection performance, with the highest AUROC score of 0.8852 at $T = 0.2$ and 0.6660 at $T = 1.0$. The consistent outperformance of our combined perturbation strategy across both temperature settings affirms its potential as a robust solution for detecting machine-authored code.

H. Impact of Perturbation Count (RQ3)

To gauge the impact of perturbation count on the efficacy of our method, we conduct experiments with varying numbers of perturbations.

Results in Table VI reveal a rapid ascent in the AUROC score as the number of perturbations increases from 10 to 20, underscoring the efficiency of our perturbation approach. Notably, an increase to 20 perturbations already yields robust

detection performance, with further increments leading to diminishing improvements. This suggests that our method requires a relatively small number of perturbations to effectively discern between human- and machine-authored code. This implies that our method is not only effective but also efficient.

I. Performance of Cross-Model Code Detection (RQ4)

	InCoder	Phi-1	StarCoder	WizardCoder	CodeGen2	CodeLlama
InCoder	0.97	0.83	0.90	0.88	0.90	0.86
Phi-1	0.72	0.86	0.77	0.79	0.76	0.68
StarCoder	0.84	0.82	0.93	0.92	0.89	0.87
WizardCoder	0.82	0.86	0.91	0.92	0.87	0.85
CodeGen2	0.59	0.83	0.65	0.69	0.85	0.68
CodeLlama	0.81	0.80	0.87	0.87	0.86	0.89

Figure 6: Cross-model detection performance

In previous sections, we primarily assess the efficacy of DetectCodeGPT within a white-box framework, where we can get access to the logits of the original code generation model, as defined in Section IV-A. In real-world scenarios, accessing the original model for code generation detection is often impractical, so we conduct cross-model detection

experiments, where we use other LLMs as surrogate models to compute the naturalness scores.

The evaluation results on *The Stack* dataset at $T = 0.2$ presented in Figure 6 highlight DetectCodeGPT’s adaptability in cross-model detection. While the algorithm excels in the white-box setting, its performance endures with only a slight reduction in cross-model application. For instance, StarCoder, when detecting code generated by WizardCoder and CodeLlama, yields AUROC scores of 0.92 and 0.87, respectively, compared to an AUROC of 0.93 when detecting its own output. We can also notice a performance decrease when detecting CodeGen2’s output. This is possibly due to the fact that CodeGen2 is trained on a more diverse dataset containing more natural language text [4]. However, Phi-1 demonstrates a relative proficiency with a score of 0.83 to detect CodeGen2’s output, which implies an ensemble of diverse detection models may enhance the system’s robustness, as suggested in [50].

These results indicate that DetectCodeGPT is a model-free method that is robust against model discrepancies, making it a viable solution for real-world applications where the source model could be unknown or inaccessible.

VI. DISCUSSION

A. Why is DetectCodeGPT Effective?

We attribute the effectiveness of our DetectCodeGPT to the following two factors:

1) *Preservation of Code Correctness*: The mask-and-recover perturbation in DetectGPT brings minor mistakes easily (e.g., misuse of an identifier), rendering the code non-functional and has negative impact on the naturalness score. Such code-cracking perturbations will violate the assumption of minimal impact on the code’s naturalness score if it is human-written in Section II-B. In contrast, inserting newlines and spaces does not affect the correctness of the code in most cases, thereby ensuring the effectiveness of our method.

2) *Emulation of Human Randomness*: As discussed in Section III-D5, human inherently exhibit less naturalness and more randomness in their use of stylistic tokens such as spaces and newlines than machines. For example, a human programmer may freely insert whitespace, especially newlines, in the code as they deem fit, whereas a machine programmer usually tries to stylize the code in a more standardized and modularized manner. Our proposed perturbation strategy mimics human’s free usage of spaces and newlines, thereby making the perturbation more “random” as is desired according to Section III.

B. Strength of DetectCodeGPT

Compared with existing methods, DetectCodeGPT eliminates the need to perturb code multiple times for each LLM and thus brings more efficiency. Compared with supervised counterparts, DetectCodeGPT distinguishes itself with a zero-shot learning capability, enabling it to detect machine-generated code without the necessity for training on extensive datasets. This model-agnostic advantage means that it can be generalized across various code LLMs.

C. Limitations and Future Directions

The main limitations of our work lie in the following two aspects: Firstly, due to the computational constraints, we only focus on a set of LLMs within 7B parameters. As the landscape of LLMs rapidly evolves, incorporating a wider array of more and larger LLMs could significantly bolster the generalizability and robustness of our findings.

Secondly, our current analysis centers exclusively on Python code, while the features of other programming languages may not be fully explored. However, based on the analysis of our method in Section VI-B, we believe that our method can be effectively generalized to other languages, especially where the functionality of code won’t be much affected after inserting newlines and spaces like C/C++, Java, and JavaScript.

Looking ahead to future work, we plan to further improve the effectiveness of DetectCodeGPT when detecting machine-generated code at higher levels of generation randomness. We note from Table IV that although DetectCodeGPT outperforms other baselines at $T=1.0$, there is still large room for improvement. Although ensembling multiple detection models may help improve the detection performance [50], we look forward to exploring more effective perturbation strategies based on code styles to further enhance the detection efficacy.

VII. RELATED WORK

A. Machine-Generated Text Detection

Recently, there has been much effort in detecting machine-generated text [7], [51]. The two main categories of detection methods are zero-shot and training-based methods, and our DetectCodeGPT falls into the former category, which eliminates the need for training data and brings more generalization ability.

As for zero-shot methods, they are usually based on the discrepancy between likelihood and rank information of human and machine’s texts [34], [33], [35]. Leveraging the hypothesis in DetectGPT [9] that machine-generated text often has a negative curvature in the log probability when the text is perturbed, many perturbation based methods have been proposed [9], [41], [52]. These methods usually perturb the text by masking a continuous span of tokens and then recover the perturbed text using another LLM like T5 [24]. The benefit of these methods is that they are zero-shot and can be applied to any LLM without access to training data. However, the perturbation process is time-consuming and computationally expensive. When it comes to the training-based methods, fine-tuning the RoBERTa [48] or T5 [24] model with data collected from different model families at different decoding settings is a common practice [53], [54], [55]. Additional information like graph structure [56], perplexity from proxy models [57] have been shown to be helpful for detection. Moreover, techniques like adversarial training [58] and contrastive learning [56] have also been proposed to improve the detection performance. The main challenge of training-based methods is that they often lack generalization ability and require access to training data from the target model [9].

B. Machine-Generated Code Detection

Research on identifying machine-generated code remains relatively scarce and is purported to be more challenging than discerning machine-generated text, according to the empirical study in [59]. GPTSniffer was first proposed to detect machine-generated code with supervised CodeBERT training [46]. Concurrent works [60] and [61] also explore perturbation-based methods for detecting machine-generated code, under similar framework to DetectGPT for text [9]. Our approach differs from these methods in that we perform a comprehensive empirical analysis of the differences between machine- and human-authored code. Based on the insights from the analysis, we proposed a innovative stylized perturbation strategy to achieve a more efficient and effective detection method.

Another related topic revolve around code watermarking techniques, which embed unique markers into the code either during the training or generation [10], [62], [63]. The detection of these watermarks subsequently enables the recognition of code generated by machines. It should be noted, however, that these watermarking methods are primarily designed to address issues related to code licensing and plagiarism [64], [65]. Their reliance on modifications to the generation model renders them unsuitable for general code detection tasks.

VIII. CONCLUSION

In this paper, we perform an in-depth analysis of the nuanced differences between machine- and human-authored code across three aspects of code including lexical diversity, conciseness, and naturalness. The results provide new insights that machines tend to write more concise and natural code, adhering to common programming paradigms, and the disparity is more pronounced in stylized tokens such as whitespaces that represent the syntactic segmentation of code. Based on these insights, we have proposed a new detection method, DetectCodeGPT, which introduces a novel stylized perturbation strategy that is simple yet effective. The experimental results of DetectCodeGPT confirm its effectiveness, demonstrating its potential to help maintain the authorship and integrity of code.

ACKNOWLEDGEMENT

This research is supported by the National Key Research and Development Program of China (Grant No. 2023YFB4503802) and the National Natural Science Foundation of China (Grant No. 62102244). We would like to thank the anonymous reviewers for their valuable feedback and suggestions.

REFERENCES

- [1] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating Large Language Models Trained on Code," Jul. 2021.
- [2] OpenAI, "ChatGPT: Optimizing language models for dialogue," Tech. Rep., 2022.
- [3] L. B. Allal, R. Li, D. Kocetkov, C. Mou, C. Akiki, C. M. Ferrandis, N. Muennighoff, M. Mishra, A. Gu, and M. Dey, "SantaCoder: Don't reach for the stars!" 2023.
- [4] E. Nijkamp, H. Hayashi, C. Xiong, S. Savarese, and Y. Zhou, "CodeGen2: Lessons for Training LLMs on Programming and Natural Languages," May 2023.
- [5] Q. Zheng, X. Xia, X. Zou, Y. Dong, S. Wang, Y. Xue, Z. Wang, L. Shen, A. Wang, Y. Li, T. Su, Z. Yang, and J. Tang, "CodeGeeX: A Pre-Trained Model for Code Generation with Multilingual Evaluations on HumanEval-X," Mar. 2023.
- [6] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov, J. Bitton, M. Bhatt, C. C. Ferrer, A. Grattafiori, W. Xiong, A. Défossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, and G. Synnaeve, "Code Llama: Open Foundation Models for Code," Aug. 2023.
- [7] X. Yang, L. Pan, X. Zhao, H. Chen, L. Petzold, W. Y. Wang, and W. Cheng, "A Survey on Detection of LLMs-Generated Content," Oct. 2023.
- [8] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, Y. Du, C. Yang, Y. Chen, Z. Chen, J. Jiang, R. Ren, Y. Li, X. Tang, Z. Liu, P. Liu, J.-Y. Nie, and J.-R. Wen, "A Survey of Large Language Models," May 2023.
- [9] E. Mitchell, Y. Lee, A. Khazatsky, C. D. Manning, and C. Finn, "DetectGPT: Zero-Shot Machine-Generated Text Detection using Probability Curvature," in *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, vol. 202, 2023, pp. 24950–24962.
- [10] T. Lee, S. Hong, J. Ahn, I. Hong, H. Lee, S. Yun, J. Shin, and G. Kim, "Who Wrote this Code? Watermarking for Code Generation," May 2023.
- [11] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," 2018.
- [12] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.
- [13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.
- [14] P. P. Ray, "ChatGPT: A comprehensive review on background, applications, key challenges, bias, ethics, limitations and future scope," *Internet of Things and Cyber-Physical Systems*, vol. 3, pp. 121–154, Jan. 2023.
- [15] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, J. Keeling, F. Gimeno, A. D. Lago, T. Hubert, P. Choy, and C. de, "Competition-Level Code Generation with AlphaCode," p. 74, 2022.
- [16] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "CodeSearchNet Challenge: Evaluating the State of Semantic Code Search," Jun. 2020.
- [17] D. Kocetkov, R. Li, L. B. Allal, J. Li, C. Mou, C. M. Ferrandis, Y. Jernite, M. Mitchell, S. Hughes, T. Wolf, D. Bahdanau, L. von Werra, and H. de Vries, "The Stack: 3 TB of permissively licensed source code," Nov. 2022.
- [18] L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima, S. Presser, and C. Leahy, "The Pile: An 800GB Dataset of Diverse Text for Language Modeling," Dec. 2020.
- [19] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, 2021, pp. 8696–8708.
- [20] D. Fried, A. Aghajanyan, J. Lin, S. Wang, E. Wallace, F. Shi, R. Zhong, S. Yih, L. Zettlemoyer, and M. Lewis, "InCoder: A Generative Model for Code Infilling and Synthesis," in *The Eleventh International Conference on Learning Representations*, Sep. 2022.
- [21] Z. Luo, C. Xu, P. Zhao, Q. Sun, X. Geng, W. Hu, C. Tao, J. Ma, Q. Lin, and D. Jiang, "WizardCoder: Empowering Code Large Language Models with Evol-Instruct," Jun. 2023.
- [22] J. Schulman, B. Zoph, C. Kim, J. Hilton, J. Menick, J. Weng, J. F. C. Uribe, L. Fedus, L. Metz, and M. Pokorny, "ChatGPT: Optimizing language models for dialogue," *OpenAI blog*, 2022.
- [23] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "LLaMA: Open and Efficient Foundation Language Models," Feb. 2023.

- [24] W. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "Unified Pre-training for Program Understanding and Generation," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, K. Toutanova, A. Rumshisky, L. Zettlemoyer, D. Hakkani-Tur, I. Beltagy, S. Bethard, R. Cotterell, T. Chakraborty, and Y. Zhou, Eds. Online: Association for Computational Linguistics, Jun. 2021, pp. 2655–2668.
- [25] H. Zhang, "Exploring regularity in source code: Software science and Zipf's law," in *2008 15th Working Conference on Reverse Engineering*. IEEE, 2008, pp. 101–110.
- [26] H. Zhang, "Discovering power laws in computer programs," *Information processing & management*, vol. 45, no. 4, pp. 477–483, 2009.
- [27] A. Hindle, E. T. Barr, M. Gabel, Z. Su, and P. Devanbu, "On the naturalness of software," *Communications of the ACM*, vol. 59, no. 5, pp. 122–131, Apr. 2016.
- [28] A. J. Albrecht and J. E. Gaffney, "Software function, source lines of code, and development effort prediction: A software science validation," *IEEE transactions on software engineering*, no. 6, pp. 639–648, 1983.
- [29] J. Rosenberg, "Some misconceptions about lines of code," in *Proceedings Fourth International Software Metrics Symposium*. IEEE, 1997, pp. 137–142.
- [30] G. K. Zipf, *Human Behavior and the Principle of Least Effort: An Introduction to Human Ecology*. Ravenio Books, 2016.
- [31] H. S. Heaps, *Information Retrieval: Computational and Theoretical Aspects*. Academic Press, Inc., 1978.
- [32] A. Barron, J. Rissanen, and B. Yu, "The minimum description length principle in coding and modeling," *IEEE transactions on information theory*, vol. 44, no. 6, pp. 2743–2760, 1998.
- [33] D. Ippolito, D. Duckworth, C. Callison-Burch, and D. Eck, "Automatic Detection of Generated Text is Easiest when Humans are Fooled," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Online: Association for Computational Linguistics, Jul. 2020, pp. 1808–1822.
- [34] S. Gehrmann, H. Strobelt, and A. M. Rush, "GLTR: Statistical Detection and Visualization of Generated Text," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, 2019, pp. 111–116.
- [35] I. Solaiman, M. Brundage, J. Clark, A. Askeff, A. Herbert-Voss, J. Wu, A. Radford, G. Krueger, J. W. Kim, and S. Kreps, "Release strategies and the social impacts of language models," 2019.
- [36] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The Curious Case of Neural Text Degeneration," in *International Conference on Learning Representations*, Sep. 2019.
- [37] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis," Sep. 2022.
- [38] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation," Oct. 2023.
- [39] B. Yetişiren, I. Özsoy, M. Ayerdem, and E. Tüzün, "Evaluating the Code Quality of AI-Assisted Code Generation Tools: An Empirical Study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT," Oct. 2023.
- [40] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, "Program Synthesis with Large Language Models," Aug. 2021.
- [41] J. Su, T. Y. Zhuo, D. Wang, and P. Nakov, "DetectLLM: Leveraging Log Rank Information for Zero-Shot Detection of Machine-Generated Text," May 2023.
- [42] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Online: Association for Computational Linguistics, Jul. 2020, pp. 7871–7880.
- [43] D. N. Manh, N. L. Hai, A. T. V. Dau, A. M. Nguyen, K. Nghiem, J. Guo, and N. D. Q. Bui, "The Vault: A Comprehensive Multilingual Dataset for Advancing Code Understanding and Generation," May 2023.
- [44] S. Gunasekar, Y. Zhang, J. Aneja, C. C. T. Mendes, A. Del Giorno, S. Gopi, M. Javaheripi, P. Kauffmann, G. de Rosa, O. Saarikivi, A. Salim, S. Shah, H. S. Behl, X. Wang, S. Bubeck, R. Eldan, A. T. Kalai, Y. T. Lee, and Y. Li, "Textbooks Are All You Need," Jun. 2023.
- [45] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim, Q. Liu, E. Zheltonozhskii, T. Y. Zhuo, T. Wang, O. Dehaene, M. Davaadorj, J. Lamy-Poirier, J. Monteiro, O. Shliazhko, N. Gontier, N. Meade, A. Zebaze, M.-H. Yee, L. K. Umapathi, J. Zhu, B. Lipkin, M. Oblokulov, Z. Wang, R. Murthy, J. Stillerman, S. S. Patel, D. Abulkhanov, M. Zocca, M. Dey, Z. Zhang, N. Fahmy, U. Bhattacharyya, W. Yu, S. Singh, S. Luccioni, P. Villegas, M. Kunakov, F. Zhdanov, M. Romero, T. Lee, N. Timor, J. Ding, C. Schlesinger, H. Schoelkopf, J. Ebert, T. Dao, M. Mishra, A. Gu, J. Robinson, C. J. Anderson, B. Dolan-Gavitt, D. Contractor, S. Reddy, D. Fried, D. Bahdanau, Y. Jernite, C. M. Ferrandis, S. Hughes, T. Wolf, A. Guha, L. von Werra, and H. de Vries, "StarCoder: May the source be with you!" May 2023.
- [46] P. T. Nguyen, J. Di Rocco, C. Di Sipio, R. Rubel, D. Di Ruscio, and M. Di Penta, "Is this Snippet Written by ChatGPT? An Empirical Study with a CodeBERT-Based Classifier," Aug. 2023.
- [47] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A Pre-Trained Model for Programming and Natural Languages," Sep. 2020.
- [48] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," 2019.
- [49] Y. Wang, H. Le, A. D. Gotmare, N. D. Q. Bui, J. Li, and S. C. H. Hoi, "CodeT5+: Open Code Large Language Models for Code Understanding and Generation," May 2023.
- [50] F. Mireshtghallah, J. Mattern, S. Gao, R. Shokri, and T. Berg-Kirkpatrick, "Smaller Language Models are Better Black-box Machine-Generated Text Detectors," May 2023.
- [51] J. Wu, S. Yang, R. Zhan, Y. Yuan, D. F. Wong, and L. S. Chao, "A Survey on LLM-generated Text Detection: Necessity, Methods, and Future Directions," Oct. 2023.
- [52] G. Bao, Y. Zhao, Z. Teng, L. Yang, and Y. Zhang, "Fast-DetectGPT: Efficient Zero-Shot Detection of Machine-Generated Text via Conditional Probability Curvature," Oct. 2023.
- [53] Y. Tian, H. Chen, X. Wang, Z. Bai, Q. Zhang, R. Li, C. Xu, and Y. Wang, "Multiscale Positive-Unlabeled Detection of AI-Generated Texts," May 2023.
- [54] H. Zhan, X. He, Q. Xu, Y. Wu, and P. Stenetorp, "G3Detector: General GPT-Generated Text Detector," 2023.
- [55] Y. Chen, H. Kang, V. Zhai, L. Li, R. Singh, and B. Ramakrishnan, "GPT-Sentinel: Distinguishing Human and ChatGPT Generated Content," 2023.
- [56] X. Liu, Z. Zhang, Y. Wang, Y. Lan, and C. Shen, "CoCo: Coherence-Enhanced Machine-Generated Text Detection Under Data Limitation With Contrastive Learning," 2022.
- [57] K. Wu, L. Pang, H. Shen, X. Cheng, and T.-S. Chua, "LLMDet: A Third Party Large Language Models Generated Text Detection Tool," Oct. 2023.
- [58] X. Hu, P.-Y. Chen, and T.-Y. Ho, "Radar: Robust ai-text detection via adversarial learning," 2023.
- [59] W. H. Pan, M. J. Chok, J. L. S. Wong, Y. X. Shin, Y. S. Poon, Z. Yang, C. Y. Chong, D. Lo, and M. K. Lim, "Assessing AI Detectors in Identifying AI-Generated Code: Implications for Education," in *2024 IEEE/ACM 46th International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*, Apr. 2024, pp. 1–11.
- [60] X. Yang, K. Zhang, H. Chen, L. Petzold, W. Y. Wang, and W. Cheng, "Zero-Shot Detection of Machine-Generated Codes," Oct. 2023.
- [61] Z. Xu and V. S. Sheng, "Detecting AI-Generated Code Assignments Using Perplexity of Large Language Models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, Mar. 2024, pp. 23 155–23 162.
- [62] Z. Sun, X. Du, F. Song, and L. Li, "CodeMark: Imperceptible Watermarking for Code Datasets against Neural Code Completion Models," in *FSE2023*, Aug. 2023.
- [63] B. Li, M. Zhang, P. Zhang, J. Sun, and X. Wang, "Resilient Watermarking for LLM-Generated Codes," Feb. 2024.
- [64] I. Cox, M. Miller, J. Bloom, and C. Honsinger, "Digital watermarking," *Journal of Electronic Imaging*, vol. 11, no. 3, pp. 414–414, 2002.
- [65] C. Collberg and C. Thomborson, "Software watermarking: Models and dynamic embeddings," in *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 1999, pp. 311–324.